

10 — Schleifen

Wiederholungen automatisieren mit while und for

HTW Berlin – Wirtschaftsingenieurwesen

SoSe 2026

Agenda

1. Wozu Schleifen?
2. `while` – Wiederholen mit Bedingung
3. Unter der Haube: der Schleifen-Zyklus & Terminierung
4. `for` – über Sammlungen iterieren
5. `range()` – Zahlenfolgen, präzise definiert
6. Das Akkumulator-Muster
7. `break` / `continue`, Dicts, Comprehensions
8. Anwendung: die Modelle aus FS 13a programmieren (Euklid/ggT)
9. **Klassiker: Zahlenraten** – live programmiert

Lernziel: Wiederholungen automatisieren – und **verstehen**, warum eine Schleife garantiert endet.

Heute: nach jedem Konzept *Live-Demo* und *Sofort ausprobieren*.

Wozu Schleifen?

Ohne Schleife

```
print(warenkorb[0])  
print(warenkorb[1])  
print(warenkorb[2])      # was, wenn 50 Einträge?
```

Mit Schleife

```
for produkt in warenkorb:  
    print(produkt)
```

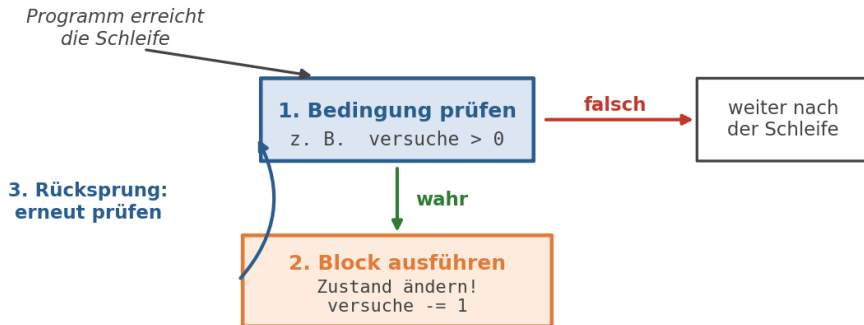
DRY wieder einmal – gleiche Logik einmal schreiben, beliebig oft anwenden.
Neu ist heute: der Rechner übernimmt das **Zählen und Wiederholen** – wir beschreiben nur noch die Regel.

while - Wiederholen mit Bedingung

```
versuche = 3                                # 1) Zustand anlegen
while versuche > 0:                          # 2) Bedingung prüfen
    print(f"Restversuche: {versuche}")
    versuche = versuche - 1                  # 3) Zustand ändern!
print("Keine Versuche mehr.")               # 4) nach der Schleife
```

- Solange die Bedingung **wahr** ist, läuft der Block
- Die Bedingung wird **vor** jedem Durchlauf geprüft – ist sie schon am Anfang falsch, läuft der Block **null Mal**
- Der Block muss den Zustand ändern, sonst ändert sich die Bedingung nie

Unter der Haube: der while-Zyklus



Formal: die Schleife wiederholt *prüfe* $B(s) \rightarrow$ *führe Block aus* (Zustand s ändert sich), bis $B(s)$ **falsch** ist. Der gesamte Speicherzustand s entscheidet – nichts anderes.

Unter der Haube: Trace - jeden Schritt mitschreiben

```
versuche = 3
while versuche > 0:
    print(versuche)
    versuche = versuche - 1
```

Schritt	versuche	versuche > 0?	Ausgabe
Prüfung 1	3	wahr	3
Prüfung 2	2	wahr	2
Prüfung 3	1	wahr	1
Prüfung 4	0	falsch	- (Schleife endet)

Diese **Trace-Tabelle** ist Ihr wichtigstes Debugging-Werkzeug: Wer eine Schleife nicht versteht, schreibt Variablenwerte pro Durchlauf auf Papier mit.

► **Live-Demo** - 01_while_countdown.py

📎 **Sofort ausprobieren** - Notebook 10, Kap. While: Countdown 10 $\rightarrow$ 0, Hochzählen 0 $\rightarrow$ 5, gerade Zahlen

Terminierung: warum hält die Schleife?

```
versuche = 3
while versuche > 0:
    print("...")
    # versuche -= 1    # vergessen -> Endlosschleife
```

- Das **Fortschritts-Argument**: eine Größe (hier versuche), die pro Durchlauf **strikt fällt** und die Bedingung irgendwann falsch macht $\Rightarrow$ die Schleife endet **garantiert**
- Ohne Fortschritt: Zustand unverändert $\rightarrow$ gleiche Prüfung, gleiches Ergebnis – **Endlosschleife**. Notbremse: **STRG+C**, in Jupyter “Interrupt Kernel”

Profi-Frage vor jedem while: “Welche Größe macht pro Durchlauf messbaren Fortschritt Richtung Abbruch?”

while True + break – bewusst eingesetzt

while True:

```
    eingabe = input("Modell? ")
```

```
    if eingabe in ["Eco-Sneaker", "Hemp-High", "Bambus-Boot"]:
```

```
        break
```

```
    print("nicht im Sortiment, nochmal.")
```

- while True ist **absichtlich** endlos – der Ausstieg wandert als break **in** den Block
- Das Fortschritts-Argument gilt trotzdem: hier ist der “Fortschritt” das Ereignis *gültige Eingabe* – ohne erreichbaren break wäre es eine echte Endlosschleife

Faustregel: jede Schleife braucht eine **erreichbare Abbruchbedingung** – im Kopf der Schleife oder als break im Block.

for - über Sammlungen iterieren

```
warenkorb = ["Eco-Sneaker", "Hemp-High", "Bambus-Boot"]
```

```
for produkt in warenkorb:  
    print(produkt)
```

- for VAR in SAMMLUNG: – in jedem Durchlauf bekommt VAR den nächsten Wert
- Funktioniert mit **Listen, Tupeln, Sets, Strings, Dicts**
- Beendet sich automatisch, wenn alle Elemente abgearbeitet sind

Unter der Haube: was for wirklich tut

```
for produkt in warenkorb:  
    print(produkt)
```

Durchlauf	Python führt intern aus	produkt ist danach
1	produkt = warenkorb[0]	"Eco-Sneaker"
2	produkt = warenkorb[1]	"Hemp-High"
3	produkt = warenkorb[2]	"Bambus-Boot"
-	Sammlung erschöpft	Schleife endet

- produkt ist eine **ganz normale Variable**, die pro Durchlauf **neu zugewiesen** wird – kein Zauber
- for ist ein while mit eingebautem Zähler: Python verwaltet Position und Abbruch selbst – **deshalb** kann ein for über eine feste Sammlung nie endlos laufen

range() - Zahlenfolgen, präzise definiert

```
for i in range(5):           # 0, 1, 2, 3, 4
    print(i)
```

Form	Werte
range(stop)	0, 1, ..., stop-1
range(start, stop)	start, ..., stop-1
range(0, 21, 5)	0, 5, 10, 15, 20

Mathematisch präzise: range(a, b, s) erzeugt die $\lceil (b - a) / s \rceil$ Werte

$$\{ a + k \cdot s \mid k = 0, 1, 2, \dots \text{ und } a + k \cdot s < b \}$$

Die obere Grenze b ist **exklusiv** - range(1, 10) endet bei 9!

► **Live-Demo** - 02_for_produkte.py, 03_for_range_summe.py

📎 **Sofort ausprobieren** - Notebook 10, Kap. For: Liste ausgeben, gegen Schwellenwert prüfen, Strings durchgehen

Das Akkumulator-Muster - und seine Invariante

```
gesamt = 0                                # 1) Startwert VOR der Schleife
for p in [89.95, 109.00, 135.50]:
    gesamt = gesamt + p                    # 2) pro Durchlauf erweitern
```

nach Durchlauf	p	gesamt
1	89.95	89.95
2	109.00	198.95
3	135.50	334.45

Die **Invariante**: nach k Durchläufen ist $\text{gesamt} = \sum_{i=1}^k p_i$ (also $p_1 + \dots + p_k$)
– das gilt in **jedem Moment**, deshalb steht am Ende garantiert die Gesamtsumme da.

break und continue

```
preise = [59.95, 89.95, 109.00, 135.50]
```

```
for preis in preise:
    if preis > 100:
        break                # Schleife komplett verlassen
    if preis == 89.95:
        continue            # Durchlauf überspringen
    print(preis)             # gibt nur 59.95 aus
```

► **Live-Demo** - 04_break_continue.py

✎ **Sofort ausprobieren** - Notebook 10, Kap. break/continue: Suche abbrechen, ungerade Zahlen überspringen

Unter der Haube: wohin springen break und continue?

Beide sind reine **Sprünge** im Schleifen-Zyklus – sie ändern nie Variablenwerte:

Anweisung	Sprungziel im Zyklus	Schleife danach
break	hinter die Schleife	beendet
continue	zurück zu Schritt 1: nächste Prüfung	läuft weiter

Trace für das Beispiel von oben:

Durchlauf	preis	Was passiert
1	59.95	beide if falsch → wird ausgegeben
2	89.95	continue → sofort nächste Prüfung
3	109.00	break → sofort hinter die Schleife

Schleife über ein Dictionary

```
preise = {"Eco-Sneaker": 89.95, "Hemp-High": 109.00}
for name, preis in preise.items():
    print(f"{name}: {preis} EUR")
```

Methode	Was kommt im Durchlauf
for k in d:	nur Schlüssel
for v in d.values():	nur Werte
for k, v in d.items():	Paare

► **Live-Demo** - 05_for_dict_warenkorb.py

List Comprehensions - unter der Haube ein for

Kurzform

```
brutto = [p * 1.19 for p in preise if p < 100]
```

Was Python daraus macht

```
brutto = []  
for p in preise:           # 1) durchlaufen  
    if p < 100:             # 2) filtern (optional)  
        brutto.append(p * 1.19)  # 3) anwenden + anhängen
```

Eine Comprehension ist **exakt diese Schleife** in einer Zeile - gleiche Reihenfolge: durchlaufen, filtern, anwenden. Nur sinnvoll, solange der Ausdruck **einzeilig** bleibt.

Rückgriff: die Modelle aus Woche 3 einlösen

In **FS 13a** haben Sie Abläufe auf Papier modelliert – Pseudocode, Flussdiagramme, Trace-Tabellen. Für zwei Bausteine fehlte damals noch die Syntax:

Baustein von damals	seit heute verfügbar
WENN ... DANN ... SONST	if / else (FS 09)
SOLANGE ... / FÜR ...	while / for (heute)

- Damit ist der Werkzeugkasten **vollständig**: Jeder Plan von damals lässt sich jetzt tippen
- Und zwar **mechanisch** – das Denken steckt bereits im Modell

Erst denken, dann tippen – heute kommt das Tippen dazu.

Unter der Haube: Übersetzen ist eine Tabelle

Pseudocode	Python	bekannt aus
ALGORITHMUS f(x)	def f(x):	FS 07
ergebnis $\leftarrow$ a + b	ergebnis = a + b	FS 04
WENN c DANN ... SONST	if c: ... else:	FS 09
SOLANGE c:	while c:	heute
FÜR i VON 1 BIS n	for i in range(1, n + 1)	heute
RÜCKGABE wert	return wert	FS 07
a MOD b	a % b	FS 05

Im **Flussdiagramm** genauso: Raute mit ja/nein $\rightarrow$ if/else, Raute mit **Rücksprung** $\rightarrow$ while, Oval "Ende" mit Wert $\rightarrow$ return.

Jede Modell-Zeile hat **genau ein** Python-Gegenstück.

Pseudocode → Code: Summe von 1 bis n

```
ALGORITHMUS SummeBisN(n)
    summe ← 0
    FÜR i VON 1 BIS n:
        summe ← summe + i
    RÜCKGABE summe
```

```
def summe_bis_n(n):
    summe = 0
    for i in range(1, n + 1):
        summe = summe + i
    return summe
```

► **Live-Demo** - 06_pseudocode_zu_code.py (Achtung: BIS n inklusiv → n + 1!) und 07_algorithmus_print_trace.py - die Papier-Trace-Tabelle vom Rechner mitgeschrieben.

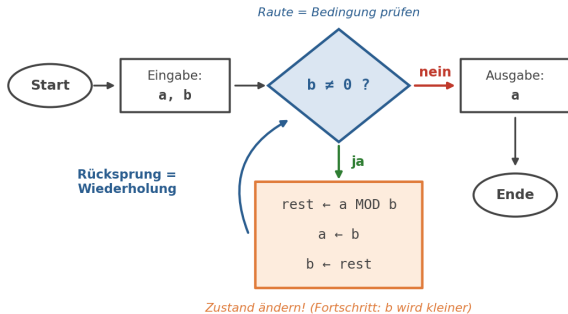
Diagramm → Code: Primzahlprüfung & Versandregel

```
ALGORITHMUS IstPrimzahl(n)
  FÜR teiler VON 2 BIS n-1:
    WENN n MOD teiler = 0 DANN
      RÜCKGABE FALSCH
  RÜCKGABE WAHR
```

```
def ist_primzahl(n):           # Randfall n < 2:
  for teiler in range(2, n):   # in der Demo
    if n % teiler == 0:
      return False             # Teiler gefunden
  return True                  # kein Teiler
```

► **Live-Demo** - 08_primzahlpruefung_schrittweise.py und 09_versand_algorithmus.py (die Versandregel aus FS 13a, über alle Tagesbestellungen ausgewertet)

Anwendung: Euklid/ggT - das Flussdiagramm von Woche 3



Erinnern Sie sich? Damals haben wir es nur **gelesen** und von Hand getraced. Heute können wir es endlich programmieren – in fünf Zeilen.

Unter der Haube: warum Euklid funktioniert - und endet

Die Kern-Invariante (für $b \neq 0$):

$$\gcd(a, b) = \gcd(b, a \bmod b)$$

- **Warum?** Der Rest ist $a - k \cdot b$: jeder gemeinsame Teiler von a und b teilt auch ihn - die Teilermenge bleibt gleich, also auch ihr Maximum
- **Fortschritt:** das neue b ist $a \bmod b$, also $< b$ und ≥ 0 - b fällt **strikt** und erreicht 0
- Bei $b = 0$ gilt $\gcd(a, 0) = a$ - deshalb am Ende return a

Genau das **Fortschritts-Argument** von vorhin - eine Größe, die strikt fällt und nicht negativ werden kann, erreicht 0.

Unter der Haube: Euklid-Trace für $\text{ggt}(48, 18)$

Prüfung	a	b	b $\neq$ 0?	rest = a % b
1	48	18	wahr	12
2	18	12	wahr	6
3	12	6	wahr	0
4	6	0	falsch	- return 6

- b fällt strikt: $18 \rightarrow 12 \rightarrow 6 \rightarrow 0$
- Der **letzte Rest vor der 0** wandert nach a – das ist der ggT

Dieselbe Tabelle haben Sie in FS 13a **von Hand** geführt. Jetzt schreibt sie der Rechner mit.

► **Live-Coding ab leerem Editor** - Referenz: 10_ggt_euklid.py (Veggie Soles bündelt 48 und 18 Kartons sortenrein - größte Bündelgröße = ggT)

Teilziele:

1. Diagramm lesen: die Raute mit Rücksprung wird zu `while b != 0:`
2. Rumpf: `rest = a % b`, dann `a = b`, `b = rest` - **Reihenfolge!**
(Variablentausch, FS 04)
3. Nach der Schleife: `return a`
4. Trace gegen die Tabelle prüfen (Ergebnis 6)
5. Als `ggt(a, b)` kapseln, beide Paare ausgeben ($48/18 \rightarrow 6$, $135/50 \rightarrow 5$)



Zusatz-Handout

-

Klassiker

13a:

ggT

(Klassiker/13a_Klassiker_GGT.pdf) - jetzt lösbar.

Klassiker: Zahlenraten - die Aufgabe

Der Klassiker der Programmierschule – mit allem von heute:

- Das Programm kennt eine **Geheimzahl** zwischen 1 und 100
- Es fragt so lange nach einem Tipp, bis die Zahl erraten ist
- Nach jedem Tipp: Hinweis **“zu groß”** oder **“zu klein”**
- Am Ende: Anzahl der benötigten **Versuche** ausgeben

So soll es laufen

Dein Tipp: 50

Zu groß!

Dein Tipp: 25

Zu klein!

Dein Tipp: 37

Richtig! 3 Versuche gebraucht.

Klassiker: Zahlenraten - Live-Coding

► **Live-Coding** ab **leerem Editor** - Referenz: 90_klassiker_zahlenraten.py

Teilziele:

1. Geheimzahl festlegen
2. while True-Frageschleife mit `int(input(...))`
3. Vergleich: `if/elif/else` → "zu groß" / "zu klein"
4. Treffer → `break`
5. Versuchszähler mitführen und ausgeben

📎 **Zum Selberlösen** - Handout "*Klassiker 10: Zahlenraten*" (Klassiker/10_Klassiker_Zahlenraten.pdf). Vertiefung: Notebook 10, Aufgabe 5.

Cheat Card

Aufgabe	Syntax
while-Schleife	<code>while cond: ...</code>
for über Liste	<code>for x in liste: ...</code>
Zahlenfolge	<code>for i in range(start, stop, step):</code>
Dict iterieren	<code>for k, v in d.items():</code>
Schleife verlassen	<code>break</code>
Durchlauf überspringen	<code>continue</code>
Liste bauen	<code>[ausdruck for x in liste if cond]</code>
Modell → Code	SOLANGE c: → <code>while c:</code> · FÜR i VON a BIS b → <code>range(a, b + 1)</code>
Anwendung heute	Euklid: <code>while b != 0:</code> + Ringtausch über rest
Klassiker heute	Zahlenraten: <code>while True</code> + <code>break</code> + Zähler

Ausblick: Notebook 11 - Fehlerbehandlung

Unser Zahlenraten hat eine Schwachstelle:

```
tipp = int(input("Dein Tipp: "))  
# ...und wenn jemand "abc" eintippt?
```

`int("abc")` → **ValueError**, Programm stürzt ab – mitten im Spiel.

Im nächsten Notebook: **try/except** – Programme robust gegen falsche Eingaben machen, statt sie crashen zu lassen.

Heute geübt

- ✓ while – bedingte Wiederholung, Prüfung **vor** jedem Durchlauf
- ✓ Der Schleifen-Zyklus: prüfen → ausführen → zurückspringen
- ✓ Terminierung: das Fortschritts-Argument
- ✓ Trace-Tabellen als Debugging-Werkzeug
- ✓ for, range() (obere Grenze exklusiv!), Akkumulator + Invariante
- ✓ break / continue, Dicts, List Comprehensions
- ✓ Modell → Code: FS-13a-Pseudocode/Diagramme übersetzt, inkl. Euklid/ggT
- ✓ **Klassiker Zahlenraten** live gesehen

Ihre Aufgaben bis zur nächsten Woche:

- Klassiker **Zahlenraten** eigenständig lösen (Handout)
- “Sofort ausprobieren”-Aufgaben im Notebook 10
- Trainingsmaterial: Quadratzahlen, FizzBuzz, Filterung